UnifiedInterpreter
HFusion/HIVM Attention 实现说明

2026 年 10 月 24 日

1 从 attention 的一次执行开始

当前 CPU 解释器读取归档的 MLIR 文件，为每条操作查找 SSA 操作数，通过 OCaml effect handler 调用 Rocq 提取的语义函数，最终得到 attention 输出矩阵和每行的 log-sum-exp。两份输入文件为：

- `artifacts/flash_attention/ir/hfusion_level.mlir`；
- `artifacts/flash_attention/ir/attention_hivm_compute.mlir`。

HFusion 文件包含 `linalg`、`hfusion` 和公共 dialect。HIVM 文件使用 `hivm.hir.*` 计算原语，并继续使用公共控制流、张量和内存操作。当前执行原语限定为这两个 attention 截面的子集，并支持在同一函数中混合使用这些原语。手工编写的 `examples/attention_mixed.mlir` 用于执行这种组合。仓库中的 H1–H5 演示程序保留在 `lib/`，attention 解释器的代码位于 `attention/`。

1.1 输入、输出与计算公式

固定配置为 batch=head=1，序列长度和 head dimension 均为 128，采用非 causal attention。输入为

$$Q, K, V \in \mathbb{R}^{128\times 128}, \qquad \alpha = 1/\sqrt{128} \ \text{（默认 scale）}.$$

对每个 query 行 i，独立数学参考实现计算

$$S_{ij} = \alpha \sum_{d=0}^{127} Q_{id} K_{jd}, \qquad m_i = \max_{0\le j<128} S_{ij}, \tag{1}$$

$$E_{ij} = \exp(S_{ij} - m_i), \qquad \ell_i = \sum_{j=0}^{127} E_{ij}, \tag{2}$$

$$O_{id} = \frac{\sum_{j=0}^{127} E_{ij} V_{jd}}{\ell_i}, \qquad L_i = m_i + \log \ell_i. \tag{3}$$

输出 O 为 128×128，输出 L 为 128 项。这些公式说明参考结果的数学含义；执行器逐条执行输入 MLIR。参考实现 `Attention.reference` 只用于结果比较，其实现见 `attention/attention.ml`。

1.2 为什么启动 program 0–19

`Attention.run` 顺序启动 20 个逻辑 program，并将编号绑定到 `%arg13`。归档 IR 的循环为：

```
scf.for %arg16 = %arg13 to %c4_i32 step %c20_i32 : i32 {
  ...
}
```

即从 program id 开始，在小于 4 时执行，步长为 20。因此 program 0–3 分别处理 query 的第 0–31、32–63、64–95、96–127 行；program 4–19 的循环体执行次数为零。编号属于 CPU 解释器的逻辑调用。同一阶段的 20 次调用共享内存，每次调用使用独立 SSA 环境。HFusion 与 HIVM 分别从独立内存状态开始执行。

1.3 一个 query 块经过哪些操作

以 program 0 为例，`memref.reinterpret_cast` 为 Q 的前 32 行建立视图，`memref.copy` 将输入复制到局部 allocation，`bufferization.to_tensor` 建立张量视图。计算过程见表 1。

计算	HFusion 文件中的操作	HIVM 文件中的操作
$Q_b K^\top$	`linalg.transpose`、`linalg.matmul`	带 `b_transpose` 的 `hivm.hir.mmadL1`
缩放与逐元素运算	`linalg.elemwise_binary`	`vmul`、`vsub`、`vadd`、`vdiv`
行最大值、行求和	`linalg.reduce`，执行显式 region	`hivm.hir.vreduce`，由 kind 指定运算
扩展行统计量	`linalg.broadcast`	reshape 与 `hivm.hir.vbrc`
指数、对数、转换	`linalg.elemwise_unary`、`hfusion.cast`	`vexp`、`vln`、`vcast`
加权 V 与写回	`linalg.matmul`、目标写回	`mmadL1`、目标写回

表 1: 归档 attention 中的操作对应关系；HIVM 简写均带 `hivm.hir.` 前缀。

IR 中还保留了由在线 softmax 形式产生的中间运算，例如由负无穷初值产生的零权重项。runner 同样执行这些操作。最终通过 `bufferization.materialize_in_destination` 写回 O 和 L 的对应行。

2 代码如何划分职责

Rocq 的 `rocq/HFusion/Instr.v` 与 `rocq/HIVM/Instr.v` 提供相应语言结构；HIVM 的 `request` 定义在 `rocq/HIVM/Execution.v`，表示已经求值的原语参数。文本执行路径使用 `attention/parser.ml` 中的操作 AST，由 runner 转成 handler 请求，再调用提取接口。

`rocq/Extraction/Extract.v` 声明导出的语义入口。Dune 将 Rocq 源文件复制到构建目录，编译后生成 `unified_semantics.ml` 和 `unified_semantics.mli`，组成 `extracted_semantics` 库。`attention/*.ml`、CLI 和 Python 前端属于手写的执行与适配代码。

部分	输入与输出	主要职责
xDSL 前端	MLIR 文本到结构化 JSON	解析 dialect、类型、属性、SSA、region；记录源码位置
OCaml AST 与验证	JSON 到类型化操作	解码字段，检查执行子集、入口签名、形状、属性与作用域
OCaml runner	操作与 SSA 环境到运行值	调度函数体、循环、归约 region；构造 effect 请求
OCaml handler	请求到提取函数调用	分派计算，适配参数，处理 continuation，提交成功的内存状态
Rocq 语义	原语参数到结果或内存 computation	定义张量遍历、广播、归约、矩阵乘法及内存行为
数值接口	标量操作到 host-float64 结果	提供加减乘除、exp/log、NaN 策略和浮点转换约定

表 2: 各部分的接口边界。

3 原语语义与数值模型

3.1 张量值、内存视图与初始化

`DefinedTensor` 携带形状、元素类型和元素列表；`EmptyTensor` 只记录形状与类型，读取元素会返回 `ReadUninitialized`。完整覆盖目标的 fill 和广播可以接受空目标；读取目标初值的矩阵累加需要已初始化张量。

`MemRefBackedTensor` 携带 descriptor。descriptor 包含 allocation 身份、offset、sizes 和 strides。对于合法逻辑坐标 $\boldsymbol{i}$，寻址形式为

$$\text{address}(\boldsymbol{i}) = \text{offset} + \sum_d i_d \ \text{stride}_d \, .$$

`to_tensor` 建立视图，后续消费通过 `materialize` 读取。这使多个视图能够引用同一 allocation。定义分别位于 `rocq/HIVM/Value.v`、`rocq/HFusion/Runtime.v`、`rocq/Memory/Model.v`。

3.2 HFusion 与 HIVM 的归约

HFusion 的 `sem_linalg_reduce` 使用目标初值和 combiner。对当前沿第二维归约的矩阵，设目标初值为 z_i，则按代码的坐标顺序计算

$$r_i = \text{fold}(f, z_i, [x_{i0}, \ldots, x_{i,n-1}]).$$

输出形状为 $[m]$。函数 f 由 MLIR reduction region 的解释执行提供。定义见 `rocq/HFusion/Semantics/Reduce.v`。

HIVM 的 `sem_reduce` 输出形状为 $[m, 1]$，从数值接口提供的 identity 开始归约。当前 host-float64 后端为 sum 选择 0，为 max 选择 $-\infty$；目标张量提供形状和类型。HIVM max 传播 NaN，

HFusion region 中的 `arith.maxnumf` 在只有一个输入为 NaN 时选择另一个输入。操作定义与数值实现分别见 rocq/HIVM/Semantics.v 和 attention/numeric.ml。

3.3 矩阵乘法、广播与转换

HFusion 的 `sem_linalg_matmul` 读取目标初值并累加。HIVM 的 `sem_mmad` 根据 `mm_init` 选择清零或读取目标，根据 `mm_b_transpose` 选择 B 的行列组织方式。当前 compute profile 要求 real M/K/N 参数为零，并从张量形状取得实际尺寸。公共的 `matrix_rows`、`matrix_columns`、`matrix_product` 在 rocq/Tensor/Matrix.v 中定义顺序遍历；每一步乘加调用参数化的标量接口。

当前 HIVM 广播支持标量扩展，以及 $[m,1] \to [m,n]$、广播维度为 $[1]$ 的形式。HFusion attention 使用的广播为 $[m] \to [m,n]$。操作分别检查输入、目标形状及类型。HFusion 新增结构操作集中在 rocq/HFusion/Semantics/Structured.v。

3.4 host-float64 的具体约定

当前工作的数值目标是验证解释过程和计算逻辑。运行时标量使用 OCaml `float`；f16/f32 标签继续参与类型检查，cast 保持浮点数值。exp/log 调用主机数学库；内积按顺序执行乘法和加法。这些选择定义了当前 CPU 数值模型。

整数操作通过 rocq/HIVM/Arith.v 处理位宽、符号扩展、除法和余数；`index` 使用该模型选择的 64 位宽度。OCaml 使用 Zarith 完成与 Rocq 整数表示之间的转换。程序入口拒绝非有限的 Q/K/V/scale；运算中的 NaN 和无穷值仍按数值接口处理，结果比较要求全部元素有限。

4 Effect handler 与 lens

4.1 四类 effect 请求

attention/handlers.ml 定义四类请求：

- `Common`：empty、reshape、整数操作及浮点标量原语；
- `HFusion`：fill、transpose、broadcast、unary、matmul、binary、cast、reduce；
- `HIVM`：调用 `Execution.evaluate` 分派的计算 request；
- `Memory`：调用 `run_memory` 执行的内存 computation。

以归档 HIVM 文件中的缩放操作为例：

```
%17 = hivm.hir.vmul
  ins(%16, %arg9 : tensor<32x128xf32>, f32)
  outs(%3 : tensor<32x128xf32>) -> tensor<32x128xf32>
```

runner 查找 %16、%arg9 和 %3，构造 BinaryRequest(VMul,...)，通过 Effect.perform 发出 HIVM 请求。handler 调用提取的 evaluate；sem_binary 检查形状和类型、扩展标量并逐元素调用 number_binary。返回张量后，runner 将结果绑定到 %17。

Effect.Deep.match_with 安装 handler。成功时通过 Effect.Deep.continue 将结果交给后续程序；语义失败通过 Effect.Deep.discontinue 传播到请求位置，由 runner 附加源码上下文。缺少对应 handler 时产生未处理 effect。

4.2 Rocq computation 与 OCaml effect 的衔接

Rocq 内存语义使用 rocq/Base/Free.v 的有限 computation：

```
Inductive computation (E : Type -> Type) (A : Type) : Type :=
| Return (value : A)
| Trigger {X : Type} (request : E X)
    (continue : X -> computation E A).
```

Return 表示结果；Trigger 保存一次请求及其后续计算。内存原语构造这样的值，由 Rocq 内存 handler 赋予状态变化。OCaml 的 Memory effect 将整段 computation 交给提取的 run_memory 执行。两处 handler 分别负责 OCaml 请求调度和 Rocq computation 的内存解释。

HFusion reduction 需要额外注意 handler 的动态作用域：提取的归约函数调用 OCaml combiner，combiner 再执行 MLIR region。runner 在该回调内部重新安装公共和内存 handler，并根据 region 内每条操作的阶段选择计算 handler。arith.maxnumf 或 arith.addf 请求使用公共语义。

4.3 逐操作选择阶段与混合执行

attention/stage.ml 定义 Common|HFusion|HIVM。OCaml parser 解码操作时填写 operation.stage：linalg.* 和 hfusion.* 标记为 HFusion，hivm.* 标记为 HIVM，支持的公共 dialect 操作标记为 Common。每个 region 内的操作同样分别标记。例如 linalg.reduce 属于 HFusion，其 region 内的 arith.addf 属于 Common。

Runner.with_operation_handlers 根据阶段启用相应的计算 handler，同时提供公共和内存 handler。外层 with_handlers 处理公共和内存请求。验证和执行入口均检查阶段与 opcode 的一致性。跨阶段执行共用 SSA 环境和内存状态，沿用同一个 memory lens。

混合示例使用 HFusion matmul 产生 %17，HIVM vmul 消费它并产生 %18，随后 HFusion reduce 消费 %18。vexp、HFusion cast、vln、输出 vcast 继续交替调用两个阶段的语义。阶段切换保留原语自身的类型、初始化和归约规则。

4.4 lens 如何限制状态修改范围

当前 OCaml 状态为 `{memory; marker}`，memory lens 读取或替换其中的内存组件。Rocq 定义的状态计算类型为

$$M(Err, S, A) = S \to \text{result}(Err, A \times S).$$

设 lens 的读写函数分别为 $g : S \to C$ 和 $p : C \to S \to S$，局部计算为 $c : C \to \text{result}(Err, A \times C)$。`zoom` 的定义可写为

$$\text{zoom}(g, p, c)(s) = \begin{cases} \text{Ok}(a, p(c', s)), & c(g(s)) = \text{Ok}(a, c'), \\ \text{Error}(e), & c(g(s)) = \text{Error}(e). \end{cases}$$

lawful lens 的三条定律为

$$g(p(c, s)) = c, \qquad p(g(s), s) = s, \qquad p(c_2, p(c_1, s)) = p(c_2, s).$$

在独立 lens 的假设下，修改内存保持另一组件的观察结果不变。`run_memory_preserves_independent` 证明这一条件性质。CPU 执行同时检查 `marker=73`，对应具体状态实例的运行检查。

`local_memory` 同时检查外层内存错误和内层复合操作错误。只有成功结果包含可提交的下一状态；OCaml handler 在解包成功结果之后才更新状态引用。这个失败原子性边界是一次 `Memory` 请求；在此之前已经成功提交的其他请求保持其结果。

4.5 尾递归内存解释器

`handle_memory_tail` 使用逆序事件累积列表。rocq/Memory/Handler.v 中的 `handle_memory_tail_correct` 证明：在相同 fuel 下，它与原 `handle_memory` 给出相同结果和内存状态，事件序列等于初始累积序列恢复顺序后接上原事件序列。`local_memory` 使用空累积列表，因此观察到的事件顺序保持一致。OCaml 的 primitive trace 另外由 `on_call` 记录；该 JSONL trace 记录提取入口和操作上下文，Rocq 内存事件由内存解释器产生。

5 解析、验证与控制流

`scripts/attention_frontend.py` 使用固定版本 xDSL 0.74.0 解析整个 module，并调用 `module.verify()`。`scripts/attention_dialects.py` 使用 IRDL 和声明式 `assembly_format` 定义归档版本所需的操作格式。公共 dialect 复用 xDSL 的实现。设备元信息作为 opaque attribute 读取。

前端保存原始 SSA 名称以及操作的文件名、偏移和行列，并导出结构化 JSON。`attention/parser.ml` 调用 Python 子进程，用 Yojson 解码为 OCaml AST。操作记录包含 opcode、stage、输入及目标、操作数类型、结果类型、属性、region、region 参数与源码位置。源码位置用于报告错误和生成 trace。

`Validate.program` 检查入口的 16 个参数、支持的操作和属性、SSA 引用、局部重复定义、结构操作的形状及类型、reshape 的重关联和 region 参数。入口绑定中 `%arg3`、`%arg4`、`%arg5` 分别为 Q/K/V，`%arg6` 为 L，`%arg7` 为 O，`%arg9` 为 scale，`%arg13` 为 program id。

`Runner.execute_block` 持有 SSA 环境并执行操作序列。`scf.for` 使用开区间上界和正步长，induction variable 的更新调用提取整数加法，并检查更新是否溢出。每个 region 建立局部参数绑定，因此不同归约 region 可以重复使用局部 SSA 名称。`linalg.yield` 返回 combiner 结果。执行预算覆盖操作及循环调度；默认 `max_steps` 为 10^6。

6 验证证据与适用边界

6.1 已经检查的性质

证据	检查对象及结论
Rocq 一般性质	内存 lens 的独立组件保持，以及尾递归内存 handler 与原定义的等价性。
Rocq 泛化示例	`AttentionStructured.v` 对指定矩阵形状、任意标量和数值接口证明遍历结果相等；检查 transpose、broadcast、fill。矩阵等式范围为 2×3 乘 3×2。
前端测试	真实归档文件、源码位置、局部 SSA、属性与形状拒绝、执行预算及循环 trace。
handler 测试	真实 float64 矩阵结果、缺少各类 handler、NaN 规则、未初始化和越界读取、非法 cast，以及复合请求中途失败时不提交状态。
端到端测试	固定种子、零 Q/K、常量矩阵三组输入，分别解释两个阶段并与独立数学参考比较。
修改操作的测试	单独的 MLIR 测试文件将 `%37` 的 vdiv 改为 vadd。前端测试检查可执行 AST 的这项差异；执行测试确认 trace 与输出随之改变。
混合执行测试	`mixed_test.ml` 检查逐操作阶段、handler 选择、跨阶段张量传递、归约回调和共享内存；`mixed_execution_test.ml` 比较混合 attention 与独立参考。

2026 年 10 月 22 日的完整验证记录显示：三组输入在两个阶段的 O 和 L 最大绝对参考误差均为零，阶段间比较通过。比较判定使用

$$|x - y| \le 10^{-10} + 10^{-9}|x|,$$

并要求两侧结果有限。2026 年 10 月 23 日编写本文时，重新读取了当前代码，并运行 `scripts/check_attention_output.py` 检查保存的结果与 trace。该检查通过输出形状、有限性、参考比较结果、program id 覆盖及调用计数检查。

2026 年 10 月 24 日，完整混合 attention 的测试与 CLI 执行均通过独立参考比较，O 和 L 的最大绝对误差均为零。逐操作 trace 确认同一次执行调用了 HFusion、HIVM 和公共语义；结果文件检查同时验证形状、program 覆盖和调用计数。小规模混合程序通过循环、归约回调和内

存读写得到预期结果 [26, 26]。同日完整执行 `duneruntest` 通过，原有三组输入在两个阶段的 O 和 L 参考误差仍均为零。

默认输入运行统计	HFusion	HIVM
runner steps	66076	592
primitive calls	33286	570
O 最大绝对参考误差	0	0
L 最大绝对参考误差	0	0

表 4: 统计来自保存的 CLI 结果；HFusion 显式执行归约 region 中的标量操作。

默认输入由种子 `[2026;1013]` 生成。两个阶段得到

$$O_{0,0} = 0.004908662752173827, \qquad L_0 = 4.853369805731775.$$

trace 覆盖 program 0–19，只有 0–3 发出 `sem_memref_copy` 请求。测试用例之间通过独立 OCaml domain 并行运行，每个用例内部仍按编号顺序启动 program。

6.2 结论可以使用到什么范围

Rocq 原语结构、所列证明和指定输入的数值一致性分别提供相应证据。当前信任边界包含 xDSL、Python 到 OCaml 的数据转换、手写 runner、handler 适配、主机数值接口、Rocq 提取及 OCaml 工具链。完整文本程序的解释正确性、HFusion 到 HIVM 的一般语义保持关系、低精度数值模型和 NPU 硬件一致性仍需独立验证。

内存模型还有明确的适用限制：当前实现拒绝重叠复制，在创建 reinterpret 视图时检查地址范围，将 empty 的非法消费报告为未初始化读取，restrict/writable 使用项目定义的内存契约。这些行为的官方来源核查和适用条件见 语义核查报告。当前 attention 使用新 allocation 作为复制目标，视图位于合法范围内，empty 在消费前由相应操作初始化。

官方规则可从仓库的 `vendor/ascendnpu-ir/README.md` 进入固定快照。其中 `HIVMVectorOps.td` 明确给出归约维度保留为 1、min/max 传播 NaN 等要求。该快照版本为 `22e3d51df4642fb0f74060af36f5c5c80be80d08`。具体操作的规范符合性需要逐项比对源码前置条件和结果定义。

7 构建、运行与代码阅读

7.1 可复现命令

在仓库根目录执行。Python 环境使用 `.venv-attention`，OCaml/Rocq 使用 `unified-interpreter` opam switch。`ATTENTION_PYTHON` 可替换为 Python 3.12 及以上解释器的完整路径。

```
ATTENTION_PYTHON=python3.12 sh scripts/bootstrap_attention.sh
export TMPDIR="$PWD/_build/task-tmp"
```

```
opam exec --switch=unified-interpreter -- dune build @all
opam exec --switch=unified-interpreter -- dune exec attention -- \
  --stage both \
  --output reports/attention.json \
  --trace reports/attention.jsonl
.venv-attention/bin/python -B scripts/check_attention_output.py
opam exec --switch=unified-interpreter -- dune runtest
```

`--stage` 接受 `hfusion`、`hivm`、`mixed`、`both`。`both` 分别运行两个归档文件；`mixed` 要求通过 `--ir` 指定包含混合操作的单个文件：

```
opam exec --switch=unified-interpreter -- dune exec attention -- \
  --stage mixed --ir examples/attention_mixed.mlir \
  --output reports/attention-mixed.json \
  --trace reports/attention-mixed.jsonl
```

`--input` 接受 JSON 对象：Q/K/V 各包含 128 行，每行 128 个数，scale 为可选有限数。`--max-steps` 指定执行预算。输出包含配置、结果矩阵、误差及调用统计。trace 中 `stage` 表示输入执行模式，`ir_stage` 表示操作阶段；宿主输入准备及结果读取的 `ir_stage` 为 null。其他字段包含 program id、循环变量、源码位置、opcode、结果 SSA 和提取入口。

构建提取库时，`extracted/dune` 依赖 Rocq 源码及 `rocq/_CoqProject`，调用 `scripts/build_extraction.sh`，在构建目录内完成 Rocq 编译和提取。因此语义源文件修改可以触发接口重新生成。

7.2 按验证问题阅读源文件

以下链接均相对于本文所在的 `docs/` 目录。

要核查的问题	文件与关键入口
实际输入和输出如何绑定	`attention/attention.ml`：`run`、`reference`、`compare_array`
MLIR 如何变成操作对象	`scripts/attention_frontend.py`：`LocatedParser`、`read_program`；`scripts/attention_dialects.py`；`attention/parser.ml`
输入子集如何验证	`attention/validate.ml`：`program`、`structured_shape`
控制流和 region 如何执行	`attention/runner.ml`：`execute_block`、`execute_op`、`with_handlers`
混合阶段如何分派	`attention/stage.ml`：`of_opcode`；`attention/runner.ml`：`with_operation_handlers`
请求如何到达 Rocq	`attention/handlers.ml`：`run`、`hfusion`、`memory_lens`；`rocq/HIVM/Execution.v`：`evaluate`、`run_memory`

要核查的问题	文件与关键入口
两层的计算定义	rocq/HFusion/Semantics/Structured.v; rocq/HFusion/Semantics/Reduce.v; rocq/HIVM/Semantics.v; rocq/Tensor/Matrix.v
内存和 lens 的保证	rocq/Base/Lens.v; rocq/Base/StateM.v: zoom; rocq/Memory/Handler.v: handle_memory_tail_correct
数值模型如何实现	attention/numeric.ml: scalar、hivm
如何重新产生证据	attention_test/parser_test.ml; attention_test/handler_test.ml; attention_test/execution_test.ml; rocq/Examples/AttentionStructured.v
导出哪些 OCaml 接口	rocq/Extraction/Extract.v; extracted/dune